# Preliminary Computational Experiments with a Genetic Algorithm for the Open Stack Problem

L. De Giovanni[♭], G. Massi[♮], F. Pezzella[♮]

[♭]Dipartimento di Matematica Pura ed Applicata

Università degli studi di Padova, via Trieste, 63 - 35121 Padova (Italy)

luigi@math.unipd.it (corresponding author)

[♮]Dipartimento di Ingegneria Informatica, Gestionale e dell'Automazione

Università Politecnica delle Marche, via Brecce Bianche - 60131 Ancona (Italy)

{massi,pezzella}@diiga.univpm.it

The *Minimization of the maximum number of Opens Stacks Problem* (MOSP) can be described as follows. We consider a cutting stock environment, where saw machines cut large sheets (standard size panels) into smaller pieces (items) of different sizes. Each panel is cut according to a determined cutting pattern, specifying a set of item types. All the panels to be cut according to the same pattern are processed consecutively. When the first panel containing a given item type is processed, a stack is reserved for that item type. The stack is closed as soon as the last panel containing that item type is processed. Given the set of item types and the set of cutting patterns, MOSP consists in determining a sequence of patterns that minimizes the maximum number of simultaneously open stacks. MOSP can be used to model problems in different application contexts: production planning, Very Large Scale Integration (VLSI) circuit design etc. [4].

In the following, we refer the the Adaptive Genetic Algorithm (AGA) presented in [2] and we give the results of a preliminary computational campaign on a variety of literature benchmarks, and on a set of real instances from woodcutting industry. AGA parameter settings are reported in [2]. Results are organized in five groups: results on aggregated instances from [8], results on individual instances from [8], results on random instances from [1], results on real instances from VLSI industry, results on real instances from woodcutting companies. AGA has been implemented in C++ and all the results have

been obtained on a personal computer equipped with a 2.1 GHz Intel Core2 processor and 2 GB RAM.

**Test benchmarks**

Test instances come from several benchmarks provided by literature and by woodcutting companies, as summarized in Table 1: the first two columns show the name of the benchmark and the number of instances. Most of literature instances are aggregated into a number of classes of homogeneous size (number of patterns and item types) and average results are available: the number of classes per benchmark is reported in the third column; if the column is empty, benchmark instances are considered individually. The fourth and the fifth columns present the minimum, average and maximum number of patterns and item types for each benchmark, and the last column gives references. We consider the following benchmarks:

- benchmarks proposed by the Constraint Modelling Challenge 2005 [8], namely:
  - 2130 random instances by Harvey. Three benchmarks are proposed (denoted "wbo", "wbop", and "wbp"), each containing 10 classes;
  - 3630 random instances by Simonis, grouped in 10 classes;
  - one class of 25 random instances by Shaw, with 20 patterns and item types;
  - 21 individual instances, one provided by Miller and 20 by Wilson (denoted GP 1–8, NWRS 1–8, SP 1–4);
- 200 individual random instances from [1];
- 11 individual instances from VLSI industry [4];
- 24 real instances from two woodcutting companies (denoted "A" and "B") available from http://www.scoop-project.net.

Table 1: Benchmarks for numerical experiments.

| **Benchmark** | **Instances** | **Classes** | **Patterns (min/avg/max)** | **Item Types (min/avg/max)** | **Ref.** |
|---|---|---|---|---|---|
| Harvey | 2130 | 30 | 10/20.8/30 | 10/20.8/30 | [8] |
| Shaw | 25 | 1 | 20 | 20 | [8] |
| Simonis | 3630 | 10 | 10/15.3/30 | 10/18.9/40 | [8] |
| Miller | 1 | | 40 | 20 | [8] |
| Wilson | 20 | | 20/56.0/100 | 10/49.5/100 | [8] |
| Chu-Stuckey | 200 | 8 | 30/71.2/125 | 30/71.2/125 | [1] |
| VLSI | 11 | | 10/41.4/141 | 10/45.7/202 | [4] |
| A | 12 | | 13/23.2/37 | 20/68.1/134 | [9] |
| B | 12 | | 10/18.8/49 | 14/24.8/60 | [9] |

**Experiments on aggregated instances from [8]**

Table 2 shows results on the benchmarks by Harvey (wbo, wbop and wbp), Simonis and Shaw. Due to the non-deterministic nature of AGA, 20 runs have been considered for each instance, and average results are compared with the Dynamic Programming (DP) approach by [3]. Each row is devoted to a class of aggregated instances: the first three columns report, respectively, the name of the benchmark, the size class (number of patterns $P$ and number of item types $I$) and the number of instances in the class itself. The following four columns show aggregated average results over the instances of each class: the best MOS found (in bold if AGA found the optimal solution for all the instances in the class), the average MOS, the standard deviation and the average running time over 20 runs. Columns "NoOpt" and $\Delta_{\max}$ show, respectively, the number of instances in the class for which AGA was not able to find the optimal solution in any of the runs, and the corresponding maximum error; last columns report the MOS and the running time of DP, able to solve to optimality all of the instances. AGA is very effective: just the 0.4% of the instances are not solved to optimality and, in these cases, the difference with the optimal solution is always as small as possible (equal to one). AGA is also very quick: running times are 1.1 seconds at most and often close to a few milliseconds. In fact, AGA initializes the population by applying a local search to several heuristic solutions, which is often able to directly find a MOS equal to the lower bound determined by a quick preprocessing. Concerning reliability, the average MOS and the standard deviation shows that all of the 20 runs reach the best MOS or a very close one (the error is always one at most).

**Experiments on individual instances from [8]**

We now focus on individual instances, that is, instances for which specific results are available from literature. Table 3 summarizes results on Miller and Wilson benchmarks

Table 2: Computational results: Harvey, Simonis and Shaw benchmarks.

| Bench. | $P \times I$ | N. | MOS | avg | dev% | T(s) | NoOpt | $\Delta_{\max}$ | DP-MOS | DP-T(s) |
|---|---|---|---|---|---|---|---|---|---|---|
| wbo | 10 × 10 | 40 | **5.9** | 5.9 | 0.00 | 0.00 | 0 | 0 | 5.9 | 0.00 |
| | 10 × 20 | 70 | **12.9** | 12.9 | 0.08 | 0.04 | 0 | 0 | 12.9 | 0.00 |
| | 10 × 30 | 100 | **20.1** | 20.1 | 0.01 | 0.08 | 0 | 0 | 20.1 | 0.00 |
| | 15 × 15 | 60 | **9.4** | 9.4 | 0.00 | 0.06 | 0 | 0 | 9.4 | 0.00 |
| | 15 × 30 | 120 | 21.0 | 21.0 | 0.06 | 0.18 | 1 | 1 | 21.0 | 0.01 |
| | 20 × 10 | 40 | **7.4** | 7.4 | 0.00 | 0.02 | 0 | 0 | 7.4 | 0.00 |
| | 20 × 20 | 90 | 13.7 | 13.7 | 0.06 | 0.22 | 1 | 1 | 13.7 | 0.01 |
| | 30 × 10 | 40 | **8.2** | 8.2 | 0.00 | 0.08 | 0 | 0 | 8.2 | 0.00 |
| | 30 × 15 | 60 | **11.6** | 11.6 | 0.00 | 0.39 | 0 | 0 | 11.6 | 0.03 |
| | 30 × 30 | 140 | 22.6 | 22.6 | 0.05 | 1.11 | 1 | 1 | 22.6 | 1.11 |
| wbop | 10 × 10 | 40 | **6.8** | 6.8 | 0.00 | 0.00 | 0 | 0 | 6.8 | 0.00 |
| | 10 × 20 | 40 | **14.3** | 14.3 | 0.07 | 0.02 | 0 | 0 | 14.3 | 0.00 |
| | 10 × 30 | 40 | **22.5** | 22.5 | 0.00 | 0.05 | 0 | 0 | 22.5 | 0.00 |
| | 15 × 15 | 60 | **10.4** | 10.4 | 0.00 | 0.05 | 0 | 0 | 10.4 | 0.00 |
| | 15 × 30 | 60 | **22.4** | 22.4 | 0.00 | 0.13 | 0 | 0 | 22.4 | 0.01 |
| | 20 × 10 | 40 | **8.1** | 8.1 | 0.00 | 0.04 | 0 | 0 | 8.1 | 0.00 |
| | 20 × 20 | 90 | **14.9** | 14.9 | 0.00 | 0.15 | 0 | 0 | 14.9 | 0.01 |
| | 30 × 10 | 40 | **8.6** | 8.6 | 0.00 | 0.06 | 0 | 0 | 8.6 | 0.00 |
| | 30 × 15 | 60 | 12.2 | 12.2 | 0.00 | 0.34 | 1 | 1 | 12.2 | 0.02 |
| | 30 × 30 | 140 | 23.9 | 23.9 | 0.03 | 0.95 | 5 | 1 | 23.8 | 0.99 |
| wbp | 10 × 10 | 40 | **7.3** | 7.3 | 0.00 | 0.00 | 0 | 0 | 7.3 | 0.00 |
| | 10 × 20 | 40 | **15.1** | 15.1 | 0.00 | 0.03 | 0 | 0 | 15.1 | 0.00 |
| | 10 × 30 | 40 | **23.2** | 23.2 | 0.06 | 0.06 | 0 | 0 | 23.2 | 0.00 |
| | 15 × 15 | 60 | **11.1** | 11.1 | 0.00 | 0.04 | 0 | 0 | 11.1 | 0.00 |
| | 15 × 30 | 60 | 23.0 | 23.0 | 0.00 | 0.14 | 1 | 1 | 23.0 | 0.01 |
| | 20 × 10 | 70 | **8.7** | 8.7 | 0.00 | 0.02 | 0 | 0 | 8.7 | 0.00 |
| | 20 × 20 | 90 | **15.4** | 15.4 | 0.03 | 0.13 | 0 | 0 | 15.4 | 0.01 |
| | 30 × 10 | 100 | **9.3** | 9.3 | 0.00 | 0.03 | 0 | 0 | 9.3 | 0.00 |
| | 30 × 15 | 120 | **13.1** | 13.1 | 0.00 | 0.18 | 0 | 0 | 13.1 | 0.01 |
| | 30 × 30 | 140 | 24.5 | 24.6 | 0.10 | 0.74 | 9 | 1 | 24.5 | 1.20 |
| Simonis | 10 × 10 | 550 | **8.0** | 8.0 | 0.01 | 0.00 | 0 | 0 | 8.0 | 0.00 |
| | 10 × 20 | 550 | **15.9** | 15.9 | 0.01 | 0.03 | 0 | 0 | 15.9 | 0.00 |
| | 10 × 30 | 550 | 24.0 | 24.0 | 0.02 | 0.05 | 1 | 1 | 24.0 | 0.00 |
| | 15 × 15 | 550 | **12.9** | 12.9 | 0.00 | 0.02 | 0 | 0 | 12.9 | 0.00 |
| | 15 × 30 | 220 | 26.0 | 26.0 | 0.02 | 0.11 | 2 | 1 | 26.0 | 0.01 |
| | 20 × 10 | 550 | **8.9** | 8.9 | 0.00 | 0.01 | 0 | 0 | 8.9 | 0.00 |
| | 20 × 20 | 220 | **18.0** | 18.0 | 0.00 | 0.11 | 0 | 0 | 18.0 | 0.01 |
| | 20 × 40 | 110 | **36.4** | 36.4 | 0.05 | 0.33 | 0 | 0 | 36.4 | 0.10 |
| | 30 × 15 | 220 | **14.0** | 14.0 | 0.00 | 0.10 | 0 | 0 | 14.0 | 0.00 |
| | 30 × 30 | 110 | **28.3** | 28.3 | 0.01 | 0.61 | 0 | 0 | 28.3 | 0.72 |
| Shaw | 20 × 20 | 25 | **13.7** | 13.7 | 0.00 | 0.34 | 0 | 0 | 13.7 | 0.01 |

from [8], related to 20 AGA runs per instance. The first two columns define instance name ad size. The following four columns are devoted to AGA and present, respectively, the best MOS found, the average running time in seconds, the success rate (SR%), that is, the number of runs leading to the best known MOS, and the maximum error (ME) over the 20 runs. Last columns report the best MOS and the running time of two dynamic programming approaches: the original one (DP) [3] and the recently proposed improved one (DP+) [1]. Optimal solutions are reported in bold: they have been certified by DP or, concerning SP3 and SP4, by DP+. AGA dominates DP with respect to the quality of the solution and is very competitive in terms of running time. In particular, AGA always finds the optimal MOS in reasonable (often negligible) running time, while the computational effort and the quality of the solutions of DP degrade with large size instances SP3 and SP4. DP+ overcomes this issue and solves instances SP2, SP3 and SP4 very quickly. Again, AGA takes negligible time for some instances, as the initial heuristic or few local search iterations find a proven optimal sequence. AGA is also very reliable, as all the 20 runs were able to find the optimal solutions, but for instances SP2 and SP4, having a success rate of 45% and 70% respectively. Anyway, in case the optimal solution is not found, AGA proposes a sequence requiring just one additional stack.

**Experiments on random instances from [1]**

Table 4 reports the comparison of AGA and DP+ on a harder benchmark by [1], containing 25 random instances (column "N") for each number of patterns "P" and item types "I": each size class contains 5 instances for each density (average number of item types per pattern) in the set {2, 4, 6, 8, 10}. Each row of Table 4 aggregates results by size (no relevant comparative difference involves density). AGA results refer to 5 run per each instance: column "Err%" show the per-cent error of the best AGA solution with respect to the optimal one, provided by DP+; "$\Delta_{\max}$" is the maximum error over all the instances in number of additional stacks; "dev%" the standard deviation, "Time" the computational time in seconds. As DP+ finds the optimal solution of all the proposed instances, the table gives just computational times (seconds): "opt" is the time to find the optimal solution, "proof" is the time to prove the optimality and "max" is the maximum time to prove optimality over the 25 instances. All the columns, but "$\Delta_{\max}$" and "max", provides average results. AGA is quite efficient (computational times never exceeds three minutes) and the error is limited on average, but may be relevant in the worst case (4.6% and up to 5 additional stacks). DP+ always finds the optimal solution and is also quite fast: small and medium intances are solved in almost no time, while proving optimality may require up to about 20 minutes for largest instances.

Table 3: Computational results: Miller and Wilson benchmarks (individual instances).

| Instance | $P \times I$ | AGA | | | | DP | | DP+ | |
|---|---|---|---|---|---|---|---|---|---|
| | | MOS | T(s) | SR% | ME | MOS | T(s) | MOS | T(s) |
| Miller | 40 × 20 | **13** | 2.68 | 100 | 0 | **13** | 0.61 | - | - |
| GP1 | 50 × 50 | **45** | 0.02 | 100 | 0 | **45** | 0.01 | - | - |
| GP2 | 50 × 50 | **40** | 0.04 | 100 | 0 | **40** | 0.01 | - | - |
| GP3 | 50 × 50 | **40** | 0.24 | 100 | 0 | **40** | 0.01 | - | - |
| GP4 | 50 × 50 | **30** | 0.02 | 100 | 0 | **30** | 0.01 | - | - |
| GP5 | 100 × 100 | **95** | 0.44 | 100 | 0 | **95** | 0.08 | - | - |
| GP6 | 100 × 100 | **75** | 0.62 | 100 | 0 | **75** | 0.14 | - | - |
| GP7 | 100 × 100 | **75** | 0.58 | 100 | 0 | **75** | 0.12 | - | - |
| GP8 | 100 × 100 | **60** | 0.58 | 100 | 0 | **60** | 0.17 | - | - |
| NWRS2 | 20 × 10 | **4** | 0.00 | 100 | 0 | **4** | 0.00 | - | - |
| NWRS1 | 20 × 10 | **3** | 0.00 | 100 | 0 | **3** | 0.00 | - | - |
| NWRS4 | 25 × 15 | **7** | 0.00 | 100 | 0 | **7** | 0.00 | - | - |
| NWRS3 | 25 × 15 | **7** | 0.00 | 100 | 0 | **7** | 0.00 | - | - |
| NWRS5 | 30 × 20 | **12** | 0.00 | 100 | 0 | **12** | 0.00 | - | - |
| NWRS6 | 30 × 20 | **12** | 0.00 | 100 | 0 | **12** | 0.00 | - | - |
| NWRS7 | 60 × 25 | **10** | 0.00 | 100 | 0 | **10** | 0.00 | - | - |
| NWRS8 | 60 × 25 | **16** | 6.00 | 100 | 0 | **16** | 2.12 | - | - |
| SP1 | 25 × 25 | **9** | 0.45 | 100 | 0 | **9** | 0.03 | - | - |
| SP2 | 50 × 50 | **19** | 10.85 | 45 | 1 | **19** | 1.65 | **19** | 0.01 |
| SP3 | 75 × 75 | **34** | 36.67 | 100 | 0 | 36 | 3600.00 | **34** | 0.41 |
| SP4 | 100 × 100 | **53** | 81.02 | 70 | 1 | 56 | 14400.00 | **53** | 9.09 |

- not available

Table 4: Computational results: random instances from [1].

| **Instances** | | | **AGA** | | | | **DP+ time** | | |
|---|---|---|---|---|---|---|---|---|---|
| **P** | **I** | **N.** | **Err%** | $\Delta_{\max}$ | **dev%** | **Time** | **opt** | **proof** | **max** |
| 30 | 30 | 25 | 0.2% | 1 | 0.00% | 1.93 | 0.00 | 0.00 | 0.00 |
| 40 | 40 | 25 | 0.4% | 1 | 0.22% | 6.58 | 0.00 | 0.00 | 0.00 |
| 50 | 50 | 25 | 1.0% | 1 | 0.22% | 17.26 | 0.00 | 0.00 | 0.01 |
| 50 | 100 | 25 | 1.1% | 1 | 0.45% | 29.74 | 0.02 | 1.63 | 8.03 |
| 75 | 75 | 25 | 2.1% | 2 | 0.56% | 54.58 | 0.01 | 0.08 | 0.63 |
| 100 | 50 | 25 | 0.7% | 1 | 0.18% | 61.83 | 0.00 | 0.00 | 0.03 |
| 100 | 100 | 25 | 3.5% | 4 | 0.96% | 118.56 | 0.06 | 2.65 | 23.11 |
| 125 | 125 | 25 | 4.6% | 5 | 0.86% | 189.98 | 2.70 | 122.60 | 1128.32 |

Table 5: Comparative results on VLSI benchmark.

| Inst. | $P \times I$ | MOS | AGA | | | CGA | | | PMA | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | SR% | ME | T(s) | SR% | ME[a] | T(s)[b] | SR% | ME | T(s)[c] |
| wli | 10 × 11 | **4** | 100 | 0 | 0.0 | 100 | 0 | 0.5 | - | - | - |
| wsn | 25 × 17 | **8** | 100 | 0 | 0.4 | 100 | 0 | 1.5 | - | - | - |
| v4050 | 16 × 13 | **5** | 100 | 0 | 0.0 | 100 | 0 | 0.5 | - | - | - |
| v4000 | 17 × 10 | **5** | 100 | 0 | 0.0 | 100 | 0 | 0.5 | - | - | - |
| v4090 | 27 × 23 | **10** | 100 | 0 | 0.5 | 100 | 0 | 2.0 | - | - | - |
| v4470 | 47 × 37 | **9** | 100 | 0 | 5.3 | 80 | 0 | 66.5 | 60 | 2 | 10 |
| x0 | 48 × 40 | **11** | 100 | 0 | 6.6 | 90 | 0 | 75.6 | 100 | 0 | 30 |
| W1 | 21 × 18 | **4** | 100 | 0 | 0.0 | 100 | 0 | 1.0 | - | - | - |
| W2 | 33 × 48 | **14** | 100 | 0 | 0.0 | 100 | 0 | 18.5 | 100 | 0 | 30 |
| W3 | 70 × 84 | **18** | 100 | 0 | 18.9 | 50 | 0 | 306.3 | 90 | 1 | 90 |
| W4 | 141 × 202 | 27 | 95 | 1 | 67.2 | 30 | 1 | 5224.7 | 20 | 7 | 2400 |

[a] referred to the best five runs
[b] on a 366 MHz Pentium II processor
[c] time limit on a 366 MHz Celeron processor
- not available

**Experiments on real instances from VLSI industry**

Table 5 and 6 are devoted to results on real instances from VLSI industry, including W4, the largest instance studied in literature, with 141 gates and 202 nets. We have considered the most competitive approaches presenting results on this benchmark, to the best of our knowledge, in particular the ones able to find the best known solution (MOS = 27) for W4: the Constructive Genetic Algorithm (CGA) [6] and the Parallel Memetic Algorithm (PMA) [5]. Note that we do not consider DP nor DP+. They both solve all VLSI instances to optimality, except W4: in this case, the available DP implementation cannot run (as the use of efficient data structures limits the maximum size of treatable instances to 128 patterns and 128 item types), while DP+ cannot find a MOS better than 28 after three hours computation.

The first columns of Table 5 report instance name, size (here, $P$ is the number of gates and $I$ the number of nets) and best known MOS (optimal values from DP and DP+ in bold). Following columns show, for each of the three approaches, the success rate (percentage of trials finding the best known results), the maximum error over all the trials and the average running times. All of the compared approaches are non-deterministic: CGA and PMA results refer to 10 trials per instance, AGA results to 20 runs. AGA outperforms both CGA and PMA in terms of success rate, as it always obtains the best known MOS, but for just one run on W4 (in this case the error is one). Other approaches have smaller success rates and, at least for PMA, larger errors. AGA seems to be competitive

Table 6: AGA computational results: computational effort for VLSI benchmark.

| Inst. | $P \times I$ | AGA | | ECS | | GRACS | | PMA | |
|---|---|---|---|---|---|---|---|---|---|
| | | SR% | FC | SR% | FC | SR% | FC | SR% | FC |
| v4470 | 47 × 37 | 100 | 573 233.9 | 60 | 169 136.0 | 100 | 90 459.8 | 60 | 176 631.0 |
| x0 | 48 × 40 | 100 | 745 765.7 | 100 | 119 296.0 | 100 | 39 187.8 | 100 | 43 033.0 |
| W2 | 33 × 48 | 100 | 46.0 | 100 | 26 185.0 | 100 | 10 002.6 | 100 | 3 523.0 |
| W3 | 70 × 84 | 100 | 990 012.5 | 50 | 540 893.0 | 90 | 372 021.0 | 90 | 203 892.0 |
| W4 | 141 × 202 | 95 | 1 019 324.4 | 55 | 1 695 924.0 | 20 | 2 357 496.0 | 20 | 9 428 591.0 |

also in terms of efficiency, even if the comparison is not fair: running times are related to different computational resources and CGA stops when the best known solution is found, while AGA and PMA do not consider this stop criterion. Results are confirmed by Table 6, where the robustness and the efficiency of AGA are further compared with the Evolutionary Clustering Search (ECS) and the Greedy Randomized Adaptive Clustering Search (GRACS) presented in [7], and, again, with PMA. In this case, a measure of efficiency is given by the number of evaluation function calls (columns FC). Concerning the success rate, as for PMA, AGA outperforms ECS and GRACS. Also, the number of function calls is limited, in particular for the large W4 instance.

**Experiments on woodcutting instances**

AGA has been applied to several real instances provided by the industrial partners of the research project "Sheet Cutting and Process Optimization for furniture enterprises" (SCOOP) developed in the sixth Framework Programme of the European Commission [9]. The dataset is available at http://www.scoop-project.net. Based on this dataset, we consider two new benchmarks of real MOSP instances, as the selection of the 12 largest instances from each of two companies (denoted by "A" and "B"). The two benchmarks have different properties (see Table 1) reflecting different production settings: company "A" uses to process a large number of patterns in the same cutting session, including many different item types (about 68 on average); company "B" supplies more standard demands and the average number of patterns and item types per instance is reduced. Results are summarized by Tables 7 and 8 for benchmarks "A" and "B" respectively. The first columns identify the instance, giving name, number of patterns ($P$) and item types ($I$), and optimal solution, if available, as from DP (Opt). Then, the results over 20 runs of AGA per instance are reported: the minimum MOS found (MOS), the percentage of trials finding the best known solution (SR%), the maximum error with respect to the best known solution (ME) and the average running time in seconds (T). Last two columns show the MOS of the sequence proposed by the Commercial Software (C.sw) used by the company and the improvement (Imp) due to AGA.

Table 7: Computational results: real instances from company "A".

| Instance | $P$ | $I$ | Opt | MOS | SR% | ME | T(s) | C.sw | Imp |
|---|---|---|---|---|---|---|---|---|---|
| A_AP-9.d_10 | 13 | 20 | 6 | **6** | 100 | 0 | 0.12 | 6 | 0 |
| A_AP-9.d_3 | 16 | 20 | 6 | **6** | 100 | 0 | 0.17 | 6 | 0 |
| A_FA+AA-_15 | 18 | 68 | 9 | **9** | 100 | 0 | 0.63 | 18 | 9 |
| A_FA+AA-_2 | 19 | 75 | 11 | **11** | 100 | 0 | 0.68 | 20 | 9 |
| A_AP-9.d_6 | 20 | 31 | 5 | **5** | 100 | 0 | 0.40 | 6 | 1 |
| A_FA+AA-_12 | 20 | 75 | 9 | **9** | 100 | 0 | 0.77 | 15 | 6 |
| A_AP-9.d_11 | 21 | 27 | 6 | **6** | 100 | 0 | 0.42 | 6 | 0 |
| A_FA+AA-_6 | 21 | 79 | 13 | **13** | 100 | 0 | 1.20 | 20 | 7 |
| A_FA+AA-_8 | 28 | 82 | 11 | 12 | 0 | 1 | 2.65 | 22 | 10 |
| A_FA+AA-_11 | 28 | 99 | 11 | **11** | 100 | 0 | 2.92 | 18 | 7 |
| A_FA+AA-_1 | 37 | 107 | 12 | **12** | 100 | 0 | 7.58 | 19 | 7 |
| A_FA+AA-_13 | 37 | 134 | - | 17 | 20 | 1 | 11.82 | 32 | 15 |

- not available

Table 8: Computational results: real instances from company "B".

| Instance | $P$ | $I$ | Opt | MOS | SR% | ME | T(s) | C.sw | Imp |
|---|---|---|---|---|---|---|---|---|---|
| B_39Q18_82 | 10 | 14 | 5 | **5** | 100 | 0 | 0.00 | 7 | 2 |
| B_42F22_93 | 10 | 18 | 5 | **5** | 100 | 0 | 0.04 | 5 | 0 |
| B_22X18_50 | 11 | 14 | 10 | **10** | 100 | 0 | 0.05 | 10 | 0 |
| B_18AB1_32 | 11 | 15 | 6 | **6** | 100 | 0 | 0.05 | 6 | 0 |
| B_CARLET_137 | 12 | 14 | 5 | **5** | 100 | 0 | 0.00 | 7 | 2 |
| B_12F18_11 | 15 | 21 | 6 | **6** | 100 | 0 | 0.16 | 6 | 0 |
| B_18CR1_33 | 18 | 20 | 4 | **4** | 100 | 0 | 0.17 | 4 | 0 |
| B_GTM18A_139 | 20 | 24 | 5 | **5** | 100 | 0 | 0.30 | 5 | 0 |
| B_23B25_52 | 21 | 29 | 5 | **5** | 100 | 0 | 0.39 | 5 | 0 |
| B_12M18_12 | 22 | 31 | 6 | **6** | 100 | 0 | 0.00 | 7 | 1 |
| B_CUC28A_138 | 26 | 37 | 6 | **6** | 100 | 0 | 0.00 | 6 | 0 |
| B_REVAL_145 | 49 | 60 | 7 | **7** | 100 | 0 | 9.37 | 9 | 2 |

The best MOS found by AGA has been certified to be optimal for all the instances but "A_FA+AA-_8" and "A_FA+AA-_13" from benchmark "A": for the former, DP provides the optimal solution and the maximum gap is just one; for the latter, the optimal solution is not available as, again, DP cannot be run (the number of item types exceeds the limit of 128). AGA shows also very reliable. The success rate is always 100% for benchmark "B" and just the two cited instances of benchmark "A" are not solved to optimality in some or all the runs: however, in these cases, the maximum error is one. Running time is often negligible and never exceeds 12 seconds for benchmark "A" and 10 seconds for "B". The comparison with commercial software shows that AGA can be successfully integrated into the production process, as the pattern sequences it suggests strongly improve the efficiency of the cutting centers. This is clear for "A", where the high number of item types limits the effectiveness of the commercial software. For example, instance "A_FA+AA-_13" required 32 stacks, meaning that the cutting process had to be frequently interrupted to allow removing pieces around the machine, or sometimes the same stack collected different item types, to be divided later on. By adopting the sequence suggested by AGA, the cutting process is still not completely automatic (the number of automated unloading stations is less than 17), but the number of interruptions has been drastically reduced. Concerning company "B", it often implemented an already optimal sequence. However, AGA improvement is crucial, as it more often allows keeping MOS within six, which is the number of available automatic unloading stations.

**Effectiveness of the initial local search**

AGA evolves an initial population generated partly at random, and partly by means of 2-OPT local search applied to several heuristic solutions (see [2] for details). Actually, this initial heuristics-driven multi-start local search has shown very effective on the tested instances, as just in a very few cases, it has not been able to directly find the optimal or best known solution: 48 of the 2130 Harvey instances, 22 of the 3630 Simonis instances, 2 instances of the woodcutting benchmark A (namely A_FA+AA-_8 and A_FA+AA-_13), 2 Wilson instances (SP2 and SP4) and the VLSI instance W4 (a total of 75 instances over the 5841 tested).

***Acknowledgements***

*This work has been financed by EU Commission, within the co-operative research project SCOOP (contract n. 032998) coordinated by Università Politecnica delle Marche – Ancona, Italy.*

*Authors wish to thanks Maria Garcia de la Banda and Peter Stuckey for providing DP source code, and Geoffrey Chu and (again) Peter Stuckey for providing support in testing DP+ algorithm.*

# References


[1] G. Chu and P.J. Stuckey. Minimizing the maximum number of open stacks by customer search. *Lecture Notes in Computer Science*, 5732:242–257, 2009.

[2] L. De Giovanni, G. Massi, F. Pezzella. An Adaptive Genetic Algorithm for Large-size Open Stack Problems. Technical Report, 2010.

[3] M. Garcia de la Banda and P.J. Stuckey. Dynamic programming to minimize the maximum number of open stacks. *INFORMS Journal on Computing*, 19(4):607–617, 2007.

[4] Y.H. Hu and S.J. Chen. GM_Plan: A Gate Matrix Layout Algorithm based on Artificial Intelligence Planning Techniques. *IEEE Transactions on Computer-Aided Design*, 9:836–845, 1990.

[5] A. Mendes and A. Linhares. A multiple-population evolutionary approach to gate matrix layout. *International Journal System Science*, 35(1):13–23, 2004.

[6] A.C.M. de Oliveira and L.A.N. Lorena. A Constructive Genetic Algorithm for Gate Matrix Layout Problems. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 21(8):969–974, 2002.

[7] A.C.M. de Oliveira and L.A.N. Lorena. Pattern Sequencing Problems by Clustering Search. *Lecture Notes in Computer Science*, 4140:218–227, 2006.

[8] B.M. Smith and I.P. Gent, editors. *IJCAI'05 – Constraint Modelling Challenge 2005*, Edimburgh, July 31, 2005. (available at `http://www.dcs.st-and.ac.uk/~ipg/challenge/ModelChallenge05.pdf`).

[9] SCOOP Team. Sheet Cutting and Process Optimization for Furniture Industry - SCOOP Project. `http://www.scoop-project.net`, 2009.